Here are some guidelines.

Be aggressive about extracting compound finder statements to their own methods, in much the same way and for much the same reason as I covered for compound Boolean logic in *Break Up Complexity*, on page 82. The methods are easier to understand and reuse if they are bound together behind a method name that defines the intent of the method. When I talk about test doubles you'll also see that having finders called behind other methods makes it much easier to avoid touching the database when you don't need to.

If a finder is extracted during refactoring and an existing test already covers its functionality, you may not need a new test to cover it. Like any other method extracted in refactoring, you aren't adding logic. Again, though, if the finder method winds up in a different class than that covered by the existing test, consider transferring the test logic.

If you're test-driving the finder method directly, you have two issues in tension. On one hand, you need to create enough objects to feel confident that the finder method is being tested. On the other, ActiveRecord finder methods need to touch the database, which makes the test slow, so you want to create as few objects as possible. Don't shy away from creating ActiveRecord objects when you're legitimately testing database retrieval behavior, but don't create more objects than you need to.

If you're testing a method that finds objects based on criteria, start with a test that creates two objects—that's one object you expect the method to find and one that you do not, which allows you to cover the logic from both sides:

```
it "finds completed tasks" do
  complete = Task.create(completed_at: 1.day.ago, title: "Completed")
  incomplete = Task.create(completed_at: nil, title: "Not Completed")
  expect(Task.complete.map(&:title)).to eq(["Completed"])
end
```

The test creates two objects and asserts that the `Task.complete` method finds only one of them. The last line of the test does something that is a little idiosyncratic but that I've found useful. Specifically, it converts the list of ActiveRecord objects (`Task.complete`) to a list of strings (`map.(&:title)`).

Generally, I'm converting a complex object to a simple one for purposes of testing. This is for increased readability—to some extent in the test, but more so in the test output. If this test fails as written, the output will look something like this:

```
  1) Failure:
Expected: ["Completed"]
  Actual: []
```